

Semester Recap for Numerical Methods in Computer Science

by Erik Girdauskas

color code: Topics, Subtopics, Definitions/Code/Observation...

→ this is based off the script and doesn't include the Python Introduction which I may cover separately

→ I will be strict and assume you have read it → if I have to explain syntax, then the syntax is new to me!

Chapter 1: Preface

Rounding Errors (lecture 1):

Def. 1.1.1: (rel., abs. error) Let $\tilde{x} \in \mathbb{R}^n$ be an approximation (approx.) of $x \in \mathbb{R}^n$. The absolute error is $\| \tilde{x} - x \|$, where $\| \cdot \|$ is a norm in $\mathbb{R}^n$. If $\tilde{x} \neq 0$, we also consider the relative error $\| \tilde{x} - x \| / \| x \|$

→ rounding errors are present in code, as formulae are not exact and represented by machine numbers

→ machine numbers are a subset of $\mathbb{R}$ that is not closed in the operations $+$, $-$, $\cdot$, $/$ → there will be rounding errors →

M , the set of machine numbers is $M = \{ d \cdot B^E, \text{ with } d = i \cdot B^{-m}, i = 0, \dots, B^m - 1, E \in \{e_{\min}, \dots, e_{\max}\} \}$

and they are of shape $x = \pm 0.a_1a_2\dots a_m \cdot B^E \rightarrow B \in \mathbb{N}$ is a base ≥ 2 , $\{e_{\min}, \dots, e_{\max}\} \subset \mathbb{Z}$ is a range of exponents, $E \in \mathbb{N}$, m is the amount of numbers for the mantisse d , condition: $a_1 \neq 0$

→ the distance between machine numbers rises with the numeric value:

→ if two positive machine numbers only differ from position $(l+1)$, $|1 - \frac{\tilde{x}}{x}| \leq 10^{-l} \rightarrow$ we want a relative error of at most $\| \tilde{x} - x \| / \| x \| \leq 10^{-l}$

Code example (under/overflow): (1.1.5) → underflow often ignored as worst case it leads to lost precision, overflow is more problematic

Code 1.1.5: Unterlauf und Überlauf

```
import numpy as np
np.seterr(under="warn")

numbers = np.array([0.5, 0.1, 0.03, 1e-16])
print(np.sum(numbers**20))

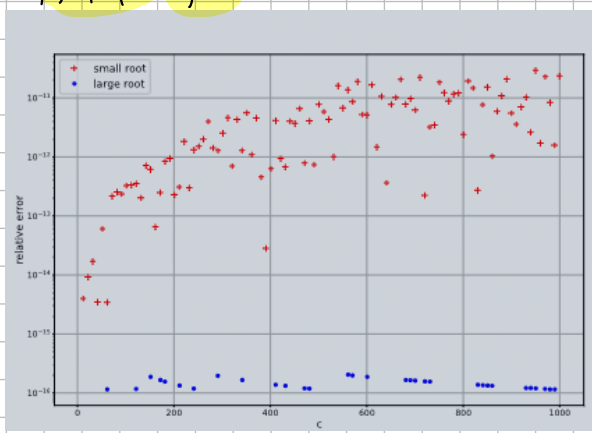
numbers = np.array([42.1, -42, -0.1])
should_be_zero = np.sum(numbers)
print(should_be_zero) # 1.4155343563970746e-15
assert should_be_zero == 0 # don't compare floats via ==
# correct way to compare floats:
tol = 1E-16 # np.finfo(float64).eps is better
s = 1. # compared to which |x| should be small
x = should_be_zero
if (abs(x) < tol*s): print(x, " is 0 for us")
else: print(x, " is not 0 for us")
x = should_be_zero/100
if (abs(x) < tol*s): print(x, " is 0 for us")
else: print(x, " is not 0 for us")

print( np.prod(10*np.arange(30)) ) # np.float64(1e+47)
print( np.uint8(23) - np.uint8(42) ) # np.uint8(237)
print(np.array([1, 2]) / 0.0) # inf
print(np.array([0, 0]) / 0.0) # NaN
```

values and plot:

Plot (1.1.6):

Code 1.1.7:
(below)



→ why is the small root more error-prone?

→ because its computation involves a subtraction instead of an addition → Obs. 1.1.9: When subtracting two similar-valued numbers with small relative errors, a small value is output but the error may add up → the error appears massive relative to the value

```
import numpy as np

def zerosquadpol(a,b):
    """
    compute the real roots of p(x) = x**2 + a*x + b
    with the standard discriminant formula
    caution: round-off!
    """
    z = np.array([])
    D = a**2 - 4*b # vectorized
    if D.any() < 0: print('no real zeros') # one of them
    else:
        wD = np.sqrt(D)
        z = np.array([ (-a-wD)/2 , (-a+wD)/2 ])

    return(z)

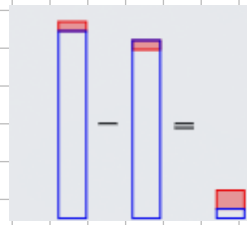
"""
Test this function for p(x) = (x-c)*(x-1/c)
a and b are computed from c, which introduces a small relative roundoff error
"""
n = 100 # how many samples
cs, cl = 2, 1000
c = np.linspace(cs, cl, num=n+1, endpoint=True) # all at once
a = -(c+1/c)
b = 1. + 0*c
z = zerosquadpol(a,b) # vectorized: all at once
z0 = z[0] # school book computed "smaller zero"
z1 = z[1] # school book computed "larger zero"
x0 = 1/c # actual "smaller zero"
x1 = c # actual "larger zero"
# plotting settings
import matplotlib.pyplot as plt
params = {'axes.labelsize': 20,
          'legend.fontsize': 20,
          'figure.figsize': (18,12),
          'xtick.labelsize': 16,
          'lines.markersize': 12,
          'ytick.labelsize': 16
        }

#-----
# plot the relative errors
plt.rcParams.update(params)
plt.semilogy(c, abs(z0-x0)/x0, 'r+', label='small root')

CHAPTER 1. VOR DEM START
10
```

```
plt.semilogy(c, abs(z1-x1)/x1, 'b.', label='large root')
plt.ylabel('relative error')
plt.xlabel('c')
plt.grid()
plt.legend()
plt.savefig('qrootsunstable.eps')
plt.savefig('qrootsunstable.pdf')
plt.savefig('qrootsunstable.png')
plt.show()
```

(catastrophic cancellation, catcan for short)



$x = 0.123467*$ ← 7. Ziffer verfälscht
 $y = 0.123456*$ ← 7. Ziffer verfälscht
 $x - y = 0.000011* = 0.11*000 \cdot 10^{-4}$ ← 3. Ziffer verfälscht

Example 1.1.11 (catcan & the differential quotient (DQ))

$\rightarrow f'(x) = \lim_{h \rightarrow 0} DQ = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$ → we can approx. $f'(x)$
 by using a small h : $f'(x) \approx \frac{f(x+h) - f(x)}{h}$ with $|h| \ll 1$

→ if the function is at least continuously differentiable twice (cont. diff.), by Taylor's Theorem, the error goes towards 0 with h :

$f(x+h) = f(x) + f'(x)h + \frac{1}{2} f''(c)h^2$ with $c = c(x,h) \in \{\min\{x, x+h\}, \max\{x, x+h\}\}$
 $\Rightarrow \frac{f(x+h) - f(x)}{h} - f'(x) = \frac{1}{2} f''(c)h \rightarrow$ test for $f(x) = e^x$,

$x \rightarrow 0 \rightarrow$ Code 1.1.12:

	$\log_{10}(h)$	relativer Fehler
	-1	0.05170918075648
	-2	0.00501670841679
	-3	0.00050016670838
	-4	0.00005000166714
	-5	0.00000500000696
	-6	0.00000049996218
	-7	0.00000004943368
	-8	0.00000000607747
	-9	0.000000008274037
	-10	0.00000008274037
	-11	0.00000008274037
	-12	0.00008890058234
	-13	0.00079927783736
	-14	0.00079927783736
	-15	0.11022302462516
	-16	1.00000000000000

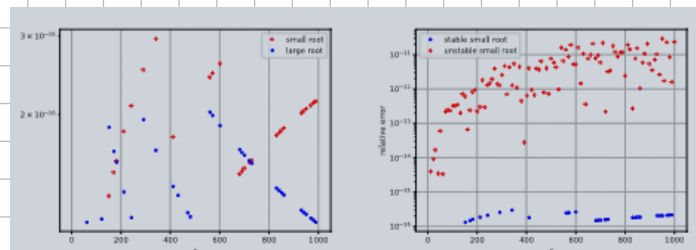
Code 1.1.12: Ableitung via Differenzenquotient

```
from numpy import exp, sin, cos

f = exp; df = exp; x = 0
print(f.__name__, x)
h = 0.1
for k in range(1,21):
    ndf = (f(x+h) - f(x))/h
    print(
        'log(h) =', -k, 'rel. err. =',
        abs((ndf-df(x))/df(x))
    )
    h *= 0.1
```

rel. error drops, then rises starkly for $h \leq 10^{-12} \rightarrow$ numpy's definition of **epsilon** is in the order of magnitude 10^{-16}
 Example 1.1.13: we will compute the zeroes of $p(x) = x^2 + tx + b$ again → avoid subtraction using **Vieta's Theorem** ($x_1 x_2 = b$)
 $\Rightarrow$ compute one root x_1 , then take $x_2 = b/x_1 \rightarrow$ Code 1.1.14:

```
def stabzerosquadpol(a,b):
    """
    compute the real roots of p(x) = x**2 + a*x + b
    with the standard discriminant formula
    avoiding cancellation
    """
    z = np.array([])
    D = a**2 - 4*b
    if D.any() < 0: print('no real zeros')
    else:
        wD = np.sqrt(D)
        z1 = np.where(a < 0, (wD-a)/2, (-wD-a)/2) # numpy array must!
        z2 = b/z1
        z = np.array([z1, z2])
    return(z)
```



Example 1.1.16 (Sum of power series): The power series of e^x converges (conv.) everywhere, so e^x can be approx'd.
 to any degree of precision anywhere by ending the sum after the power series' first terms

Code 1.17:
 (below)

```

import numpy as np
def expeval(x, tol=1e-8):
    y = 1.; term = 1. # first term
    k = 1
    while abs(term) > tol*y:
        term *= x/k # next summand
        y += term # summation
        k += 1
    return y

xs = np.arange(-20, 21, 2)
ys = np.array([expeval(x) for x in xs])
zs = np.exp(xs) # actual integrated exponential
A = np.array([ys, zs, abs(ys-zs)/zs])
print(A.T)

def println(line):
    opt = ""; opt += str(int(line[0])); opt += " & "
    opt += '{:.12e}'.format(line[1]); opt += " & "
    opt += '{:.12e}'.format(line[2]); opt += " & "
    opt += '{:.16f}'.format(line[3])
    return(opt)

print(" \\\n".join([println(line) for line in A.T]))

```

x	Approximation $\text{exp}(x)$	$\text{exp}(x)$	$ \text{exp}(x) - \text{exp}(x) $
-20	6.147561824192e-09	2.061153622439e-09	1.9825830337278934
-18	1.598372035943e-08	1.522997974471e-08	0.0494905855000887
-16	1.124750330014e-07	1.125351747193e-07	0.0005344259515302
-14	8.315441787438e-07	8.315287191036e-07	0.0000185918296272
-12	6.144210514234e-06	6.144212353328e-06	0.0000002993214527
-10	4.539992960354e-05	4.539992976248e-05	0.000000035010444
-8	3.354626281246e-04	3.354626279025e-04	0.000000006620042
-6	2.478752175842e-03	2.478752176666e-03	0.0000000003325188
-4	1.831563887901e-02	1.831563888873e-02	0.000000005307235
-2	1.353352831996e-01	1.353352832366e-01	0.0000000002736031
0	1.000000000000e+00	1.000000000000e+00	0.0000000000000000
2	7.389056095384e+00	7.389056098931e+00	0.000000004799685
4	5.459814992815e+01	5.459815003314e+01	0.0000000019230582
6	4.034287929504e+02	4.034287934927e+02	0.0000000013442484
8	2.980957980774e+03	2.980957987042e+03	0.0000000021025843
10	2.202646574759e+04	2.202646579481e+04	0.00000000021437994
12	1.627547911384e+05	1.627547914190e+05	0.0000000017238449
14	1.202604279794e+06	1.202604284165e+06	0.0000000036341346
16	8.886110500976e+06	8.886110520508e+06	0.00000000021979897
18	6.565996891074e+07	6.565996913733e+07	0.0000000034509718
20	4.851651930671e+08	4.851651954098e+08	0.0000000048287371

→ we see; for large negative x our approx. is horrid → we can prevent this by using $e^x = 1/e^{-x}$ for $x < 0$

Example 1.18 (Differentiation with imaginary (Im.) step); to avoid catcan with the previous approx., use the purely im. step in $\mathbb{C}$, assuming that f is very smooth around x_0 and as a function of a complex var. $z \in \mathbb{C}$ (analytical around x_0). $\Rightarrow f$ can be written as the sum of a power series that conv. in a complex circle around x_0 : $f(z) = \sum_{j=0}^{\infty} a_j (z-x_0)^j$ for all z with $|z-x_0| < \rho$ for fixed $\rho > 0$, $a_j \in \mathbb{R}$ → f is infinitely diff. around x_0 and its derivatives are of type $f^{(n)}(x_0) = n! a_n \in \mathbb{R}$

→ we can write Taylor's Theorem with an imaginary step: $f(x_0 + ih) = f(x_0) + f'(x_0)ih - \frac{1}{2} f''(x_0)h^2 - \frac{i}{6} f'''(x_0)h^3 + O(h^4)$ for $\mathbb{R} \ni h \rightarrow 0$ → $f(x_0)$ and $-\frac{1}{2} f''(x_0)h^2$ are real, so the imaginary part of the equation is: $\text{Im}(f(x_0 + ih)) = h f'(x_0) - \frac{1}{6} f'''(x_0)h^3 + O(h^5)$

For $\mathbb{R} \ni h \rightarrow 0 \Rightarrow f'(x_0) = \lim_{h \rightarrow 0} \frac{\text{Im}(f(x_0 + ih))}{h} + \frac{1}{6} f'''(x_0)h^2 + O(h^4)$ for $\mathbb{R} \ni h \rightarrow 0$ (no subtraction!) $\Rightarrow f'(x_0) \approx \frac{\text{Im}(f(x_0 + ih))}{h}$

Image 1.1.19: (here you see how more smoothness allows a formula without catcan)

Example 1.1.20 (Richardson, important)

Idea: use a method converging so fast that we don't even reach the area of cancellation → task: approximate a smooth function (fct.) f in a point x :

First Taylor expansion terms: $f(x+h) = f(x) + f'(x)h + \frac{1}{2} f''(x)h^2 + \frac{1}{6} f'''(x)h^3 + \frac{1}{24} f^{(4)}(x)h^4 + \dots$

and $f(x-h) = f(x) - f'(x)h + \frac{1}{2} f''(x)h^2 - \frac{1}{6} f'''(x)h^3 + \frac{1}{24} f^{(4)}(x)h^4 - \dots$

$$\rightarrow f(x+h) - f(x-h) = 2f'(x)h + \frac{1}{3} f'''(x)h^3 + \frac{1}{60} f^{(5)}(x)h^5 + \dots \Rightarrow f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{1}{60} f'''(x)h^2 - \frac{1}{420} f^{(5)}(x)h^4 - \dots$$

$\Rightarrow$ we have an approximation converging in $O(h^2)$ using the symm. $d_0(h) = \frac{f(x+h) - f(x-h)}{2h}$ → still suffers from catcan!

→ Observe that $f'(x) - d_0(h) = -\sum_{k=1}^{\infty} \frac{f^{(2k+1)}(x)}{(2k+1)!} h^{2k}$, a sum of only even powers of h

→ Idea: write this for another val. of h → $f'(x) - d_0(h/2) = c_1 \frac{1}{2} h^2 + c_2 \frac{1}{2^3} h^4 + \dots + c_n \frac{1}{2^n} h^{2n} + O(h^{2n+2}) \Rightarrow$ linear combination (LC)

eliminating the h^2 term: $f'(x) - \frac{4d_0(h/2) - d_0(h)}{3} = \bar{c}_2 h^4 + \bar{c}_3 h^6 + \dots + \bar{c}_n h^{2n} + O(h^{2n+2})$, $\bar{c}_2, \dots$ are the new coeff. of the powers

of h → $d_1(h) = \frac{4d_0(h/2) - d_0(h)}{3}$ (explanation: $4f'(x) - 4d_0(h/2) = c_1 h^2 + \dots$, $(4f'(x) - 4d_0(h/2)) - (f'(x) - d_0(h)) = (c_1 h^2 + \dots)$

$(c_1 h^2 + \dots)$ → then divide $3f'(x)$ by 3 to get $d_1(h)$

repeat: $f'(x) - d_1(h) = \bar{c}_2 h^4 + \bar{c}_3 h^6 + \dots + O(h^{2n+2})$, $f'(x) - d_1(h/2) = \bar{c}_2 \frac{1}{2^4} h^4 + \bar{c}_3 \frac{1}{2^6} h^6 + \dots + O(h^{2n+2}) \rightarrow$ another LC yields

$f'(x) - \frac{16d_1(h/2) - d_1(h)}{15} = \bar{\bar{c}}_3 h^6 + \dots + O(h^{2n+2}) \rightarrow$ the error of $d_2(h) = \frac{16d_1(h/2) - d_1(h)}{15}$ conv. like $h^6 \rightarrow$ repeat as far as

smoothness allows it $\Rightarrow$ for $k=1,2,\dots,k_i$ $R_{f,k} = \frac{h^k R_{f,k-1} - R_{f,k-1}}{4k-1} \rightarrow F(x) - R_{f,k} = C \cdot (h/2^k)^{2k+2}$

$\rightarrow$ Richardson scheme:

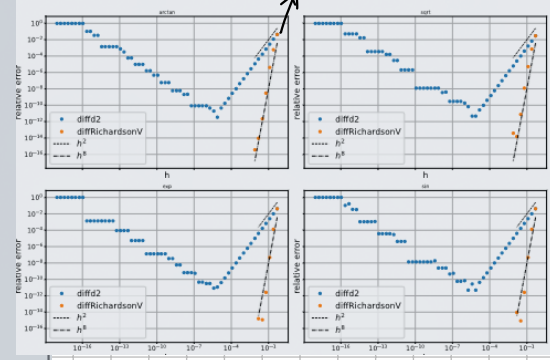
Code 1.1.21: (Richardson)

Img. 1.1.22: catan still happens, but later

ℓ	$k=0$	$k=1$	$k=2$	$k=3$
0	$d(h) = R_{0,0}$			
1	$d(h/2) = R_{1,0} \rightarrow R_{1,1}$			
2	$d(h/2^2) = R_{2,0} \rightarrow R_{2,1} \rightarrow R_{2,2}$			
3	$d(h/2^3) = R_{3,0} \rightarrow R_{3,1} \rightarrow R_{3,2} \rightarrow R_{3,3}$			
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

```
import numpy as np

# Richardson extrapolation: fixed level for vectorisation
def diffRichardsonV(f, x, h0, rtol=1e-12, atol=1e-12):
    nit = 30 # max depth of iterations
    # first column at once
    h = h0/2*np.arange(nit)
    fp, fm = f(x+h), f(x-h)
    y = (fp-fm)/2/h
    # go column by column in Richardson
    # we prefer to store it in the last part of the vector
    for j in range(1,nit):
        fact = 4**j
        y[j:] = (fact*y[j:] - y[j-1:-1])/(fact-1)
        errest = abs(y[j]-y[j-1])
        print(j, h[j], errest)
        if errest < rtol*abs(y[j]) or errest < atol:
            break
    #return y[j:], h[j:]
    return y[j+1:], h[j+1:] # return the last computed to show cancellation
```



Computational Effort:

Def. 1.2.1:

The computational effort of a numeric algorithm is the amount of elementary ops. (add/subtract...)

$\rightarrow$ today, this is not an accurate descriptor of expected runtime $\rightarrow$ memory bandwidth is more decisive today

Def. 1.2.2: The asymptotic complexity of an alg. describes its worst case dependence on one or more problem size parameters (like input vector length $n \rightarrow \infty$ for lin. algebra)

Def. 1.2.3: we write $F(n) = O(G(n))$ for two functions $F, G: \mathbb{N} \rightarrow \mathbb{R}$ if there exists a constant $C > 0$ and $n \in \mathbb{N}$ s.t. $F(n) \leq C \cdot G(n)$ for all $n \geq n$ (you will need no further explanation of this)

$\rightarrow$ warnings for numpy: force a deep copy when working with ndarrays when needed (otherwise an array meant to not be mutated can contain new values), never use numpy.vectorize $\rightarrow$ writes a slow loop around code

$\Rightarrow$ to vectorize, set a hard upper bound for iterations and use ndarrays of that dimension $\rightarrow$ cost: wasted memory

Code examples:

```
import numpy as np
n = 4
A = np.diag(np.mgrid[1:n+1])
A[:, -1] = A[:, 1:] = np.mgrid[1:n+1]
print('A=\n', A)
B = A[:, :-1]; # B is not a copy, but another view on A
print('B=\n', B) # note: rows of A in reversed order
A[2,2] = -2
print('A=\n', A) # changed an element on the diagonal of A
print('B=\n', B) # the corresponding element "of B" changed!
B[2,2] = -10
print('B=\n', B) # changed an element on the diagonal of B
print('A=\n', A) # the corresponding element "of A" changed!
print('B.base=\n', B.base)
```

Code 1.2.5: Übungen mit Indizierungen

```
"""https://numpy.org/doc/stable/user/basics.indexing.html#advanced-indexing"""
x = np.arange(10) # 0,1,2,...,9
print(x[0], x[-1], x[-2]) # 0, 9, 8
print(x.reshape(2,5)) # makes a copy
print(x) # no change
x.shape = (2,5) # modify the attribute shape of this x
print(x) # see: now x is 2-dimensional
print(x[0]) # 0 refers now to the first row in x
# x[0] is a view, not a copy:
x[0][2] = -2; print(x)
x[0,2] = 2; print(x) # more efficient than x[0][2]
# be caution with the data type!
x[0,2] = 2.5; print(x) # 2.5 was cast to the data type of x (int)
# basic slicing returns a view
y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print(y[1:7:2]) # ([1, 3, 5])
print(y[-2:10], y[-2:1]) # [8, 9]
print(y[-3:3:-1]) # [7, 6, 5, 4]
print(y[2], y[2:3]) # watch the difference!
print(x)
print(x[1:10:5, :-1]) # is the same as the more general:
obj = (slice(1, 10, 5), slice(None, None, -1)); print(x[obj])
x = np.array([[1],[2],[3]], [[4],[5],[6]])
```

```
print(x.shape); print(x[1:2])
print(x[... , 0]) # is the same as
print(x[:, 0])
print(x[:, np.newaxis, :, :].shape) # is synonym to:
print(x[:, None, :, :].shape)
# advanced indexing returns a copy
x = np.arange(10, 1, -1); print(x)
print(x[np.array([3, 3, 1, 8])]) # [7, 7, 9, 2]
y = np.arange(35).reshape(5, 7); print(y)
print(y[np.array([0, 2, 4]), np.array([0, 1, 2])])
print(y[np.array([0, 2, 4]), 1]) # elements [0,1], [2,1], [4,1]
print(y[np.array([0, 2, 4]), :]) # copies of the rows 0,2,4 of y
# select corners of a 4x3 array:
x = np.arange(12).reshape(4,3); print(x)
rows = np.array([0,0], [3,3])
cols = np.array([0,2], [0,2])
print(x[rows, cols])
# shorter via broadcasting:
rows = np.array([0, 3])
cols = np.array([0, 2])
print(x[rows[:, np.newaxis], cols])
print(x[rows[:, np.newaxis], cols])
# even shorter:
print(x[np.ix_(rows, cols)])
print(np.ix_(rows, cols))
# note the difference:
print(x[rows, cols])
```

Matrix arithmetic:

$\rightarrow$ we consider a vector $x \in \mathbb{C}^m$ a $m \times 1$ matrix, x^T if we want it in row form

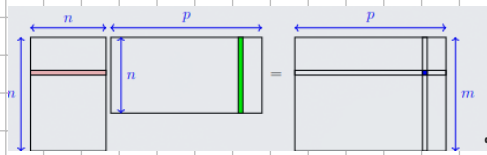
if you want it in row form and complex conjugate, write x^H

$\rightarrow$ scalar product for $a, b \in \mathbb{C}^m$: $\langle a, b \rangle = \bar{a} \cdot b = a^H b = \sum_{k=1}^m \bar{a}_k b_k \in \mathbb{C}$

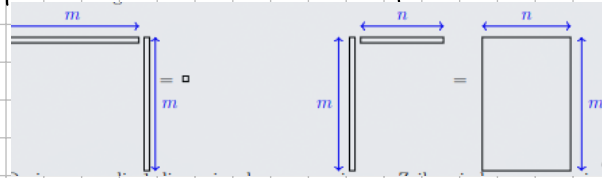
$\rightarrow$ tensor (dyadic) product of $a \in \mathbb{C}^m, b \in \mathbb{C}^n$: $ab^H = [a_i \bar{b}_j] \in \mathbb{C}^{m \times n}$

$\rightarrow$ these are special cases of a matrix product of $A \in \mathbb{C}^{m \times n}, B \in \mathbb{C}^{n \times p}$: $AB = [\sum_{j=1}^n a_{ij} b_{jk}] \in \mathbb{C}^{m \times p}$

Valid:



Special cases (scalar and tensor product)



→ Code (warning: 1D arrays are always rows in numpy):

Code 1.3.1: Verschiedene Arten, das Skalarprodukt zu berechnen

```
a = np.array([1., 2., 3.]) # 1. ensures data type is float
b = np.array([4.j, 5.j, 6.j]) # 1j = complex imaginary unit
print(a*b) # use broadcasting: array as result, not a scalar product
print((a*b).sum(), np.sum(a*b)) # object functions should be faster
print(b @ a) # no conjugation
print(np.dot(b,a)) # no conjugation, better style
print(np.vdot(b,a)) # conjugation of first argument
print(np.dot(b.conj(), a)) # manual conjugation; b.conj() makes no copy
print((b.conj()*a).sum(), np.sum(np.conjugate(b)*a)) # last makes a copy
print(np.tensordot(b, a, axes = 1)) # same as dot
print(np.tensordot(b.conj(), a, axes = 1)) # same as vdot
# look up np.einsum
print(np.einsum('i,i',b,a)) # np.dot(b,a)
print(np.einsum('i,i',b.conj(),a)) # np.vdot(b,a)

# make them 2D ndarrays; caution on type and shape of output
A = a.reshape(1,3); print(A) # view, no copy
B = b.reshape(1,3); print(B)
print(A @ B)
# print(A @ B) """mismatch"""
print(B @ A.T) # real scalar product of b and a
print((B.conj() @ A.T)) # complex scalar product of b and a
print(A @ B, A @ B.conj())
```

→ einsum can be faster/more RAM efficient than @/sum/transpose, but slower than dot/inner/tensordot

→ use outer for the outer product kron for Kronecker product (each of first array scales second array), I am not covering einsum now

Obs. 1.3.3: The result of mult. of a $m \times n$ matrix A with a vector x is an LC of the cols. of A : $Ax = \sum_{j=1}^n x_j A_{:,j}$

Obs. 1.3.4: The rank of the result of MM (matrix mult.) is less than or equal to the min. of the rank of the factors: $\text{Rank}(AX) \leq \min(\text{Rank}(A), \text{Rank}(X))$

→ each col. of the result B of the MM $AX=B$ is an LC of the cols. of A :

$$A [x_{:,1} \ x_{:,2} \ \dots \ x_{:,p}] = [b_{:,1} \ b_{:,2} \ \dots \ b_{:,p}]$$

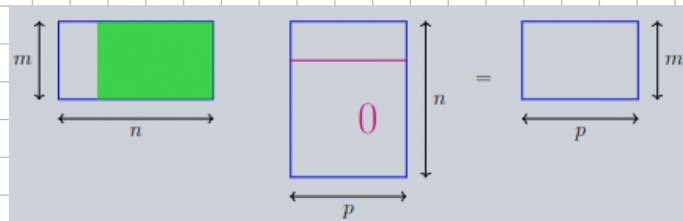
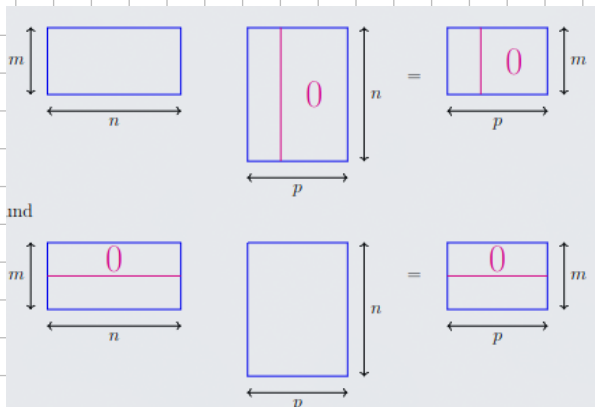
$\Rightarrow Ax_{:,1} = b_{:,1}$ → coeff. in this LC are entries in cols. of X → e.g.:

$$Ax_{:,2} = b_{:,2} \quad b_{:,1} = \sum_{j=1}^n x_{j,1} A_{:,j} \rightarrow \text{Rank}(B) \leq \text{Rank}(A)$$

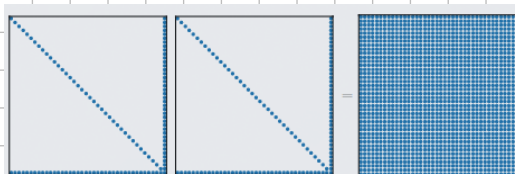
$Ax_{:,p} = b_{:,p}$ → analogous argumentation for rows leads to result

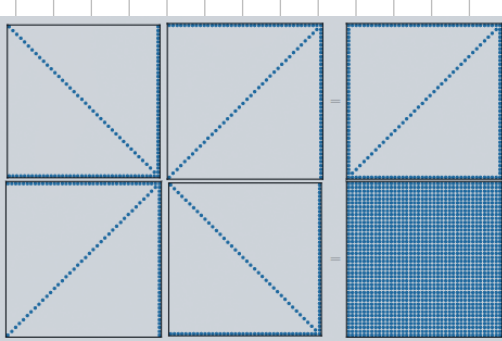
Obs. 1.3.5: Formula 1.3.9. shows the result of MM as a matrix constructed from cols. $b_{:,j} = Ax_{:,j}$ while formula 1.3.10 shows it as a matrix from rows $b_{i,j} = A_{i,:} x$ → each can be advantageous depending on structure

Parts multiplying zero blocks are irrelevant for result → you can save memory:

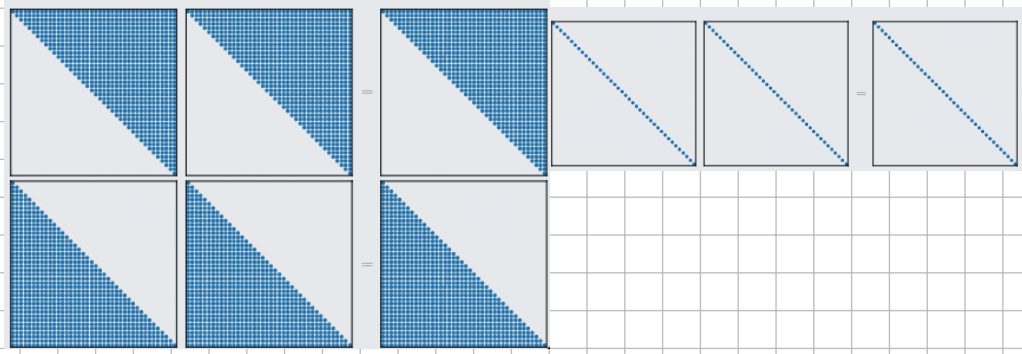


→ This rule is very important for sparse matrices like arrow matrices:





Obs. 1.3.6: The set of diag./upper triangular/lower triangular matrices are closed off w.r.t. matrix multiplication:



Obs. 1.3.7: recall the difference between mult. with a diag. matrix from the left and the right: mult. from the left scales rows:

$$\begin{bmatrix} d_1 & 0 & 0 \\ 0 & d_2 & 0 \\ 0 & 0 & d_n \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 a_{11} & d_1 a_{12} & \dots & d_1 a_{1m} \\ d_2 a_{21} & d_2 a_{22} & \dots & d_2 a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ d_n a_{n1} & d_n a_{n2} & \dots & d_n a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{A}_{1,:} \\ \vdots \\ d_n \mathbf{A}_{n,:} \end{bmatrix}$$

mult. from right scales cols:

$$\begin{bmatrix} d_1 & 0 & 0 \\ 0 & d_2 & 0 \\ 0 & 0 & d_n \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 a_{11} & d_1 a_{12} & \dots & d_1 a_{1m} \\ d_2 a_{21} & d_2 a_{22} & \dots & d_2 a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ d_n a_{n1} & d_n a_{n2} & \dots & d_n a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{A}_{:,1} \\ \vdots \\ d_n \mathbf{A}_{:,n} \end{bmatrix}$$

Example 1.3.9:

Let A be a fully randomized $n \times n$ matrix and D

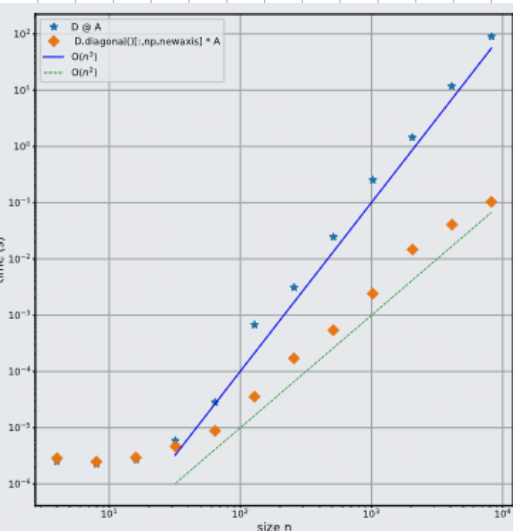
a $n \times n$ diag. matrix $\rightarrow$ mult. DA using " $D @ A$ " in numpy

costs $O(n^3)$ ops., but if you exploit the sparseness of D by only scaling rows of A with the corresponding entries of D , this takes $O(n^2) \rightarrow$ avoid loops and don't create copies; use $D.\text{diagonal}()[i, np.newaxis] * A$

$\rightarrow$ assume D is given $\rightarrow$ we need a view/shallow copy of the diag. with D , $\text{diagonal}()$ and we newaxis to broadcast

$\rightarrow$ runtimes:

Obs. 1.3.11: you can measure runtimes like this if `timeit` is unavailable in python:



```
import time

nrEXP = 10 #number of experiments
ts = [] # measured times
for k in range(nrEXP):
    start = time.process_time()
    result = func() # call the function/part we measure
    end = time.process_time()
    ts.append(end - start)
print(f'Min. Time taken: {min(ts):.6f} seconds')
```

$\rightarrow$ measures wall time $\rightarrow$ influenced by concurrently running processes

$\rightarrow$ you can also use a decorator for this:

```
import time

def mytimeit(rep=1): # decorator with argument
    def mytimeit_real_decorator(func): # actual decorator
        def wrapper(*args, **kwargs):
            ts = [] # measured times
            for k in range(rep):
                start = time.process_time()
                result = func(*args, **kwargs)
                end = time.process_time()
                ts.append(end - start)
            print(f'Min. Time taken: {min(ts):.6f} seconds in {func.__name__}')
            wrapper.ats.append(min(ts)) # record the minimal measured time
            return result
        wrapper.ats = [] # declare attribute to use later
        return wrapper
    return mytimeit_real_decorator
```

Obs. 1.3.14: (showing row/col. ops.

via MM) let i, j and consider the

op. $B_{i,:} = (-2)A_{i,:} + A_{i,:} \rightarrow$ left mult. with elementary matrix $E^{(i)}$ its elements are identical to I , but element (i,i) below the diag. is -2 , for example:

$$E^{(2,1)}A = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ -2a_{11} + a_{21} & -2a_{12} + a_{22} & -2a_{13} + a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

→ mult. from the right uses the rows of A:

$$AE^{(2,1)} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a_{11} - 2a_{12} & a_{12} & a_{13} \\ a_{21} - 2a_{22} & a_{22} & a_{23} \\ a_{31} - 2a_{32} & a_{32} & a_{33} \end{bmatrix},$$

→ meaning $B_{i,j} = A_{i,j} + (-2) \cdot A_{i,j-1}$, as $i > j$, for $i < j$, $B_{i,j} = (-2) \cdot A_{i,j-1} + A_{i,j}$

$$AE^{(1,2)} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} 1 & -2 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a_{11} & -2a_{11} + a_{12} & a_{13} \\ a_{21} & -2a_{21} + a_{22} & a_{23} \\ a_{31} & -2a_{31} + a_{32} & a_{33} \end{bmatrix}$$

Obs. 1.3.15: The product of two matrices can be computed **blockwise** → recall MM is not commutative.

Let dimensions $M, N, K \in \mathbb{N}$ and block lengths $1 \leq n \leq N$ ($n' := N - n$), $1 \leq m \leq M$ ($m' := M - m$), $1 \leq k \leq K$ ($k' := K - k$) →

consider matrices $A_{11} \in \mathbb{R}^{m,n}$, $A_{12} \in \mathbb{R}^{m,n'}$, $A_{21} \in \mathbb{R}^{m',n}$, $A_{22} \in \mathbb{R}^{m',n'}$, $B_{11} \in \mathbb{R}^{n,k}$, $B_{12} \in \mathbb{R}^{n,k'}$, $B_{21} \in \mathbb{R}^{n',k}$, $B_{22} \in \mathbb{R}^{n',k'}$, used as blocks to form $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \in \mathbb{R}^{m,n}$, $B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \in \mathbb{R}^{n,k}$ → AB can be computed the same way as the product of 2×2 matrices:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix} \quad (1.3.11)$$

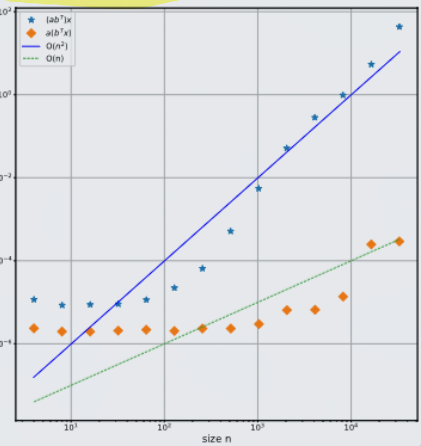
Complexity table:

Name	Operation	#{*,/}	#{+,-}	Komplexität
Skalarprodukt	$x \in \mathbb{R}^n, y \in \mathbb{R}^n \rightarrow x^T y$	n	n-1	$O(n)$
Tensorprodukt	$x \in \mathbb{R}^m, y \in \mathbb{R}^n \rightarrow xy^T$	nm	0	$O(mn)$
Matrix × Vektor	$x \in \mathbb{R}^n, A \in \mathbb{R}^{m,n} \rightarrow Ax$	mn	(n-1)m	$O(mn)$
Matrixprodukt	$A \in \mathbb{R}^{m,n}, B \in \mathbb{R}^{n,p} \rightarrow AB$	mnp	(n-1)mp	$O(mnp)$

→ Obs. 1.3.16: for $m=n=p$, the MM product can be computed in $O(n \log^2 2)$ and $O(n^{2.36})$ using Strassen's algorithm

Example 1.3.17: rank 1 matrices can always be written as a **tensor product** of 2 vectors; $y = (ab^T)x$ costs $O(mn)$ ops., as the full $m \times n$ matrix ab^T multiplies x → meanwhile, $y = a(b^T x)$ costs $O(mn)$ ops., as $b^T x$ is the scalar product

Compute times:



Example 1.3.19: Let $A, B \in \mathbb{R}^{n,n}$ with $p \ll n$ (low rank), let R be the upper right part of AB^T , direct mult. Rx (implemented as $np.triu(A @ B.T) @ x$) costs $O(pn^2)$ ops.

→ visualization: (brackets show order of ops.) → most expensive part is mult. with special matrix T :

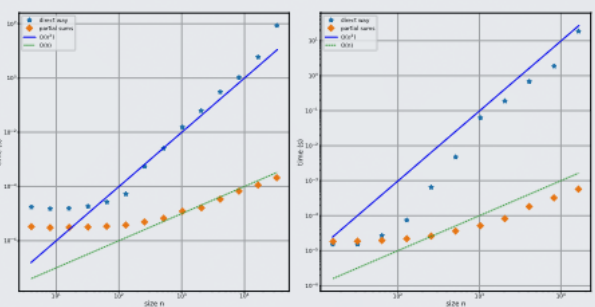
$$y = \text{triu}(ab^T)x = \begin{bmatrix} a_1 b_1 & a_1 b_2 & \dots & a_1 b_n \\ 0 & a_2 b_2 & \dots & a_2 b_n \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & a_n b_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$$= \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 0 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 1 \end{bmatrix} \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

upper triangular of only 1's → mult. of T with a vector is simple:

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ 0 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} s_n \\ s_{n-1} \\ \vdots \\ s_1 \end{bmatrix}, \quad s_j := \sum_{k=n-j+1}^n v_k$$

⇒ result contains **partial sums** of els. of inverted vector v → they can be computed with $v.\text{cumsum}()$ or $np.\text{cumsum}(v)$ efficiently → can be organized to take $O(n)$ instead of $O(n^2)$ ops. → for p cols. $O(n \cdot p)$ ops.; $AB^T = \sum_{k=1}^p A_{:,k} (B_{:,k})^T$, much cheaper than $O(pn^2)$ for the direct implementation → compute times:



Defo 1.3.21: The Kronecker product $A \otimes B$ of $A \in \mathbb{R}^{m,n}$, $B \in \mathbb{R}^{k,l}$ with $m, n, k, l \in \mathbb{N}$ is the $(ml) \times (nk)$ matrix

$$A \otimes B := \begin{bmatrix} (A)_{1,1}B & (A)_{1,2}B & \dots & (A)_{1,n}B \\ (A)_{2,1}B & (A)_{2,2}B & \dots & (A)_{2,n}B \\ \vdots & \vdots & \ddots & \vdots \\ (A)_{m,1}B & (A)_{m,2}B & \dots & (A)_{m,n}B \end{bmatrix} \in \mathbb{R}^{ml,nk}$$

Example 1.3.22 (mult. of Kronecker product with vector) Constructing $A \otimes B$ then multiplying with the vector x of length nk yields

complexity $O(m \cdot n \cdot l \cdot k) \rightarrow$ by splitting x into n blocks of equal length: $x = [x_1 \ x_2 \ \dots \ x_n]^T$ with $x_j \in \mathbb{R}^k$, we can

directly compute the result:

Compute times:

$$(A \otimes B)x = \begin{bmatrix} A_{1,1}Bx^1 + A_{1,2}Bx^2 + \dots + A_{1,n}Bx^n \\ A_{2,1}Bx^1 + A_{2,2}Bx^2 + \dots + A_{2,n}Bx^n \\ \vdots \\ A_{m,1}Bx^1 + A_{m,2}Bx^2 + \dots + A_{m,n}Bx^n \end{bmatrix}$$

Chapter 2: polynomial Interpolation

